



Transcript

Week 24: Natural Language Processing

Video 1 - Natural Language Processing

What if a machine could truly understand what you're saying—and even what you mean? That's the promise of Natural Language Processing, the tech behind everything from smart assistants to real-time translation. It's how AI learns to speak our language—and it's changing the world!

In this module, we'll explore the basics of Natural Language Processing—what it is, why it matters, and how it's used in the real world. You'll learn how to clean and prepare raw text using standard preprocessing techniques. We'll then look at tokenisation, a key step that breaks text into smaller pieces for analysis. Finally, we'll uncover how word embeddings help machines understand the deeper meaning and relationships between words. By the end, you'll have a solid foundation in essential NLP concepts.

Ready to train machines to understand human language—just like we do? Let's break down real-world text, uncover meaning with smart techniques, and explore how NLP powers the tools we use every day.

Video 2 - What is Natural Language Processing (NLP)?

Don't just watch—start thinking like a machine. Use unsupervised learning to uncover hidden patterns in unlabelled data and see how algorithms learn without guidance. Let's dive in and try it yourself.

Natural Language Processing, or N-L-P, is a fascinating branch of Artificial Intelligence that focuses on enabling machines to understand, interpret, and generate human language. Its primary goal is to bridge the gap between how people communicate and how machines process information.

At its core, N-L-P helps machines understand and generate language in a way that feels natural to us. For example, when you ask Siri to set a reminder or use Google Translate to convert a sentence, N-L-P is working behind the scenes to process your input and respond appropriately.

N-L-P plays a critical role in enhancing human-machine communication. It powers tools like voice assistants, translation apps, and customer support bots that can interpret natural speech and respond in real time. These systems depend on a combination of language understanding and data processing—two key components that N-L-P brings together.

In this lesson, you'll explore how NLP makes it possible for machines to interact with us more intelligently—by understanding not just words, but their meaning and intent. This capability is transforming how we use technology every day.

Natural Language Processing is essential because it's transforming how we interact with technology. Consider how Google Translate helps travellers navigate unfamiliar places, or how chatbots can answer customer queries and schedule appointments—these are real-world examples of N-L-P in action.

N-L-P also plays a critical role in accessibility. Voice-to-text tools enable people with hearing impairments to communicate more easily by converting spoken language into



written text in real time. This improves access to technology and information for many users.

In business settings, N-L-P is used to extract valuable insights from text. For instance, companies analyse customer reviews to understand public sentiment or identify common issues. These insights can guide product improvements or service strategies.

Another major advantage of N-L-P is automation. It handles repetitive tasks like email filtering, spam detection, and chatbot responses, freeing up time and resources.

With its ability to support innovation, improve accessibility, extract meaning from language, and automate tasks, N-L-P continues to be a driving force in today's AI-powered world.

Natural Language Processing is now widely used across many industries to solve real-world problems using human language.

In **healthcare**, NLP helps analyse unstructured medical records and assists doctors or nurses by powering chatbots that support patients, answer common queries, or help with scheduling appointments. This makes healthcare more efficient and responsive.

In **e-commerce**, NLP enhances online shopping by generating personalised product recommendations and analysing customer reviews through sentiment analysis. This helps businesses better understand customer preferences and improve the shopping experience.

In the **finance** sector, NLP is used to detect fraudulent transactions and analyse customer feedback. Banks and financial institutions use it to monitor sentiment, identify issues, and deliver more personalised services.

In **entertainment**, NLP is behind features like automatic subtitle generation and podcast transcription, helping make media more accessible and searchable.

As the technology continues to grow, we can look forward to even more human-like conversations and real-time language translation that bridges global communication gaps. This is the evolving power and promise of NLP.

Natural Language Processing has come a long way since the 1960s. In its early stages, NLP relied on rule-based systems, where grammar and syntax were manually defined using fixed rules. This approach formed the foundation of machine language understanding.

One of the first major breakthroughs during this time was **ELIZA**, an early chatbot developed to simulate human-like conversation using simple pattern matching. Although basic, it was a significant step in showing how machines could interact with people using language.

As the field progressed, the focus shifted to **statistical methods**. Techniques like **Hidden Markov Models** brought major improvements to areas such as speech recognition by using probabilities and real-world data, rather than just hand-coded rules.

Today, we're in the era of **deep learning**. Modern N-L-P models use neural networks to process and generate human-like language. For instance, **BERT** helps machines understand words in context—recognising that the meaning of a word changes based on the words around it. Meanwhile, **GPT-4** pushes the boundaries even further by generating coherent, fluent responses that resemble human conversation.

This rapid evolution has made NLP more accurate, more scalable, and more powerful than ever before—opening up exciting possibilities for the future. And this is only the beginning.

Can you think of a way to apply NLP in your daily life—maybe to improve your search



results, summarise your notes, or even spot fake news? Try exploring one real-world NLP tool and reflect on how it shapes the way we communicate with machines.

Video 3 - What is Text Preprocessing?

How can a machine understand human language if the text is full of noise, typos, and inconsistencies? Text preprocessing is the first step that cleans and organises raw text so machines can make sense of it. Why is this so important in NLP? Let's find out.

Text preprocessing is the essential first step in any Natural Language Processing task. Raw text is often messy—full of typos, irrelevant symbols, mixed cases, and inconsistent formats. If we don't clean it up, it can confuse our models and lead to poor predictions.

Let's say we're analysing tweets. We might see hashtags like hash symbol followed by a word, mentions starting with the at symbol, emojis, random capitalisation, and extra punctuation marks. Some of these may be useful, but many just add noise.

Text preprocessing helps remove this clutter. Common tasks include converting all letters to lowercase using the `.lower()` function, removing punctuation like full stops, commas, and exclamation marks, and handling special characters such as @, #, or &.

By cleaning and structuring text, we make it easier for our models to find meaningful patterns. These simple steps can significantly improve model accuracy, reduce training time, and help systems understand language better.

Stopwords are common words in a sentence—like *is*, *the*, and *on*—that don't add much value during analysis. They appear frequently but carry very little meaning on their own. Removing them helps reduce noise, shrink the dataset size, and focus only on meaningful words that matter to our model.

For instance, consider the sentence:

"The cat is sitting on the mat."

This version contains several stopwords.

After removing them, we get:

"Cat sitting mat."

Notice how the core meaning stays intact, even though the sentence is shorter. We often use built-in functions like `nltk.corpus.stopwords.words("english")` in Python to filter these out automatically. This step makes the data cleaner and more useful for downstream NLP tasks like classification or summarisation.

Stemming is like taking a shortcut to reach the base form of a word. It removes suffixes to standardise words, so terms like *running* and *runner* both reduce to *run*. This helps models treat similar words as the same, which improves efficiency in tasks like text classification.

However, stemming isn't always linguistically accurate. For example, it might turn *flies* into *fli*, which isn't a valid word. That's because stemming focuses on speed, not precision.

Despite this drawback, stemming is still useful in many NLP tasks where perfect language understanding isn't critical. Popular algorithms for stemming include the **Porter Stemmer** and **Snowball Stemmer**, both available in Python libraries like NLTK.

Lemmatisation is a more accurate technique than stemming. Instead of simply chopping off word endings, it looks at the context and converts each word into its correct dictionary form, known as the *lemma*. This ensures the base form is linguistically valid.

For instance, *running* is converted to *run*, and more significantly, *better* becomes *good*. A



stemmer wouldn't catch that.

Lemmatisation is ideal when preserving meaning is important. Tools like the **Word Net Lemmatizer** or **spaCy** use built-in linguistic knowledge to produce precise results, making this method especially useful for tasks like sentiment analysis or translation.

Text preprocessing often follows a series of simple but essential steps. Let's take an example sentence:

"The QUICK brown foxes were JUMPING!!!"

First, we convert all text to lowercase:

"the quick brown foxes were jumping!!!"

Next, we remove punctuation and special characters—like exclamation marks and extra symbols.

Then, we eliminate stopwords such as *the*, *were*, and *was*.

After that, we apply either stemming or lemmatisation.

- *Stemming* might reduce *jumping* to *jump*, or even *jump* to *jump*.
- *Lemmatisation* would turn *jumping* into the correct dictionary form: *jump*.

The result is a clean and standardised sentence like:

"quick brown fox jump"

This streamlined version is now ready for more advanced NLP steps, such as tokenisation, embedding, or model training.

Can you build your own text preprocessing pipeline? Try cleaning a messy sentence by applying lowercase conversion, removing punctuation, cutting out stopwords, and then using stemming or lemmatisation. How does your cleaned version compare to the original—and why might it perform better in an NLP task?

Video 4 - Introduction to Text Tokenisation

How does a machine begin to understand a sentence? It starts by breaking the text into smaller, manageable units called tokens. This process, known as tokenisation, is the foundation for everything else in Natural Language Processing.

Tokenisation is one of the first and most important steps in any Natural Language Processing task. It breaks down a sentence into smaller, manageable units called *tokens*, making it easier for a machine to process.

For example, the sentence:

"Natural Language Processing is fun!"

can be split into word tokens like:

["Natural", "Language", "Processing", "is", "fun", "!"]

or even into character tokens such as: N-A-T-U-R-A-L

These tokens act like building blocks that help the model understand language structure. Whether we split by word, subword, or character, tokenisation shapes how text is represented and analysed in N-L-P.

Tokenisation can happen at different levels depending on the task. The most common is **word tokenisation**, where a sentence like *"I love NLP"* becomes three tokens: *["I", "love", "NLP"]*. It's quick and works well for general analysis.



When we need more detail, we use **subword tokenisation**. This splits words into smaller parts—like “unbelievable” into [“un”, “believe”, “able”]. It helps when models come across unfamiliar words or spelling variations.

The most granular method is **character tokenisation**. For example, the word “AI” becomes [“A”, “I”]. This is useful in deep learning tasks or when processing rare or complex scripts.

Each method has its own purpose. Choosing the right one depends on how precise or flexible your NLP model needs to be.

There are different ways to break text into words, depending on how precise we need to be. The simplest method is **whitespace tokenisation**, where text is split wherever there’s a space. For example, “NLP is exciting” becomes “NLP”, “is”, “exciting”.

But sometimes, we need to be more careful. **Punctuation-aware tokenisation** keeps punctuation marks as separate tokens. For instance, “Hello comma world exclamation mark” is split into “Hello”, “comma”, “world”, “exclamation mark”. This helps preserve the structure and meaning of the sentence.

For more advanced control, we use **library-based tokenisers** like those in **N-L-T-K**, **spaCy**, or **Hugging Face**. These tools can handle edge cases like contractions and complex punctuation, making them ideal for real-world language data.

Subword tokenisation is a powerful method used when dealing with rare or unfamiliar words in a dataset. Instead of ignoring these words or treating them as unknown, it breaks them into smaller, meaningful units. This helps the model understand their structure and retain important parts of the meaning.

Take the word “unbelievable”. Rather than treating it as one long token, subword tokenisation breaks it down into [“un”, “believe”, “able”]. Similarly, “unknowingly” becomes [“un”, “know”, “ing”, “ly”]. This approach not only captures the semantics but also helps reduce the overall size of the vocabulary the model needs to learn.

Techniques like **Byte Pair Encoding (BPE)** and **WordPiece Tokenisation** are commonly used in modern NLP models like BERT and GPT. These methods allow machines to handle new or complex words more effectively, making them ideal for tasks like translation, summarisation, and question answering.

Character tokenisation breaks text into its smallest units—individual characters. This means that the word “NLP” would be represented as [“N”, “L”, “P”]. It’s a very fine-grained method that can capture subtle patterns in language.

This approach is especially useful when working with noisy data, low-resource languages, or words with complex spelling variations. It’s also commonly used in tasks like **spelling correction**, where knowing the exact sequence of letters matters.

Character tokenisation is valuable in **morphologically rich languages**, where single words can contain a lot of embedded meaning. However, since it produces longer sequences, models need more steps to process the same amount of text—so it must be used carefully in large-scale tasks.

Choosing the right tokenisation method depends on your task, the language you're working with, and the type of model you're using. If you're doing a general NLP task like text classification or sentiment analysis, **word tokenisation** is often a simple and effective choice.

For deep learning models like BERT or GPT, **subword tokenisation** works better. It



breaks words into smaller parts like prefixes and suffixes, helping the model handle unfamiliar words more efficiently.

When dealing with noisy data, rare spellings, or highly complex languages, **character tokenisation** is the best option. It splits words into individual characters, offering the highest level of detail.

Understanding these trade-offs helps you choose the method that gives your model the best chance of success. With tokenisation complete, you're now ready to move on to representing text in numerical form through word embeddings.

Now that you've explored different tokenisation methods, can you choose the right one for a real-world task? Try taking a sentence and applying word, subword, and character tokenisation. Which one do you think works best for your task, and why?

Video 5 - Introduction to Word Embeddings

How can machines understand that *king* and *queen* are related, or that *happy* and *joyful* mean nearly the same thing? Word embeddings make this possible by turning words into numbers that capture meaning, not just spelling. This step takes us from symbolic text to meaningful maths—powering everything from search engines to chatbots.

Word embeddings are a way to represent words using numbers—but not just any numbers. These are **vectors** that capture a word's meaning, and even its relationship with other words.

For example, the connection between “*king*”, “*man*”, “*woman*”, and “*queen*” can be expressed mathematically. If you subtract the word vector for “*man*” from “*king*”, and then add “*woman*”, the result is close to the vector for “*queen*”. That's:

king – man + woman ≈ queen

This approach allows machines to understand not only the meaning of individual words, but also the subtle relationships between them—making it a foundational step for many NLP tasks.

Word embeddings work by representing similar words with similar vectors. For example, “*cat*” and “*dog*” may appear close together in vector space, while “*cat*” and “*car*” would be further apart because they're less related.

To generate these embeddings, we start by building a vocabulary from the text data. Each word is assigned a unique vector, which is then refined using a neural network. The network learns from patterns in the data to place related words near each other in the embedding space.

These embeddings are created by training a neural network on a large collection of text, allowing the model to learn and encode meaningful relationships between words. This approach helps machines understand analogies like “*king minus man plus woman equals queen*,” and significantly improves the performance of NLP tasks such as translation, classification, and sentiment analysis.

Let's explore how machines understand the meaning of words using *word embeddings*.

We start with **Word-2-Vec**, which uses two main methods: Continuous Bag of Words, or *C-BOW*, and *Skip-Gram*. *C-BOW* predicts a word based on its surrounding words, while *Skip-Gram* does the reverse—predicting surrounding words from a given word. For example, given the phrase “the cat sat on the dash,” *C-BOW* might predict the missing



word is “mat”.

Next is **GloVe**, which stands for *Global Vectors*. It learns word relationships by looking at how often words appear together in a massive text corpus. So, words like “king” and “queen” may end up close together in the embedding space because they appear in similar contexts.

Then we have **Fast-Text**, which improves upon Word-2-Vec by breaking words into smaller parts called *subword units*. This helps with words the model hasn't seen before. For instance, if it knows “run” and sees “running,” it can still make a useful prediction.

Finally, **contextual embeddings** like **BERT** go a step further. They generate different embeddings for the same word depending on the sentence. For example, “bank” will have different meanings in “river bank” versus “financial bank”—and BERT captures that difference.

These embeddings help machines understand not just words, but their meanings in context.

Word-2-Vec is a technique that helps machines understand the meaning of words by turning them into numbers—called *embeddings*. It uses two main approaches: **C-BOW**, or *Continuous Bag of Words*, and **Skip-Gram**.

In **C-BOW**, we try to guess the missing word using the words around it. Think of the sentence: “The cat is dash on the mat.” C-BOW would learn to predict that the missing word is “sitting”.

In contrast, **Skip-Gram** does the opposite. It starts with a known word, like “sitting”, and tries to predict the surrounding words: “The”, “cat”, “is”, “on”, “the”, “mat”.

Both methods use **neural networks** to learn, and the goal is to minimise errors during training. C-BOW is faster and works well with frequent words. Skip-Gram, though slower, is better at handling rare or unusual words.

Together, they form the foundation for how Word-2-Vec creates useful word representations that machines can work with.

GloVe, which stands for *Global Vectors*, is a method for turning words into numbers that represent meaning—called *word embeddings*. Unlike Word-2-Vec, which focuses on local context, GloVe looks at the big picture. It captures how often words appear together across a large set of text.

Here’s how it works:

First, GloVe builds a **co-occurrence matrix**. This is a large table that records how frequently words appear near one another. For instance, words like “king” and “queen” might both appear often with “royalty”. That high co-occurrence tells the model there’s a strong relationship between these words.

Next, GloVe uses a mathematical process called **matrix factorisation** to transform that data into word embeddings. These embeddings help machines understand that “king”, “queen”, and “royalty” are semantically related, even if they don’t appear in the same sentence.

This global view makes GloVe especially powerful when we need a broader understanding of word relationships.

Let’s look at how machines understand the meaning of words in different contexts.



First, we have **static embeddings**, used in models like Word2Vec and GloVe. These assign a single, fixed vector to each word — no matter where or how the word is used. That means the word “*bank*” will always have the same representation, whether you're talking about a riverbank or a financial institution.

This approach struggles with **polysemy**, where words have multiple meanings, and it doesn't adapt well to the structure or context of real-world language.

To overcome this, we use **contextual embeddings**. These models, such as **BERT** and **GPT**, generate different word embeddings depending on the sentence.

For example, in the sentence “*He stood by the river bank,*” the word “*bank*” means something completely different from “*She deposited money in the bank.*” Contextual models understand this and give each word a meaning that fits its sentence.

This makes them far better at handling ambiguity, capturing grammar, and interpreting meaning in a more human-like way.

Now that you've seen how word embeddings capture meaning, can you think of everyday sentences where the same word changes meaning depending on context? Try identifying them—and think about how a machine might learn to tell the difference.

Video 6: Natural Language Processing- Demo Part 1

Hello and welcome everyone. In this section, we are going to talk about one of the most important foundations in natural language processing. We are going to talk about text preprocessing.

Now, when we work with human language, we deal with text that comes in various forms from various sources. You may be getting the text as customer reviews or a social media post, a headline, an entire book corpus. When you observe this raw text, they are messy and have all sorts of inconsistencies, extra characters, random casing.

And also you will observe that there are many words which do not carry useful meaning for our task. But unlike humans, computers do not automatically understand context or meaning, they rely entirely on the way we feed them the data. So, if the input text is inconsistent, the computer will treat similar words as completely unrelated, and this eventually affects the quality of any predictions we try to make.

That is where text processing comes in and solves this problem. So, when I say text processing, it means cleaning up the text and putting it into a standard form so that a machine can identify, find patterns from it more easily. Let us understand this with few simple examples.

Imagine we have the words dog, dog and dog. To a computer, these are three different tokens, but to us they mean exactly the same thing. Lower casing all the text makes sure that these variations are treated as one word.

Another example is punctuations. Punctuation marks like commas, periods or exclamation points are often not useful for analysis unless we are doing something like sentiment detection where an exclamation might add some meaning. But in most tasks, we remove them to reduce noise.

So, these are the first two steps which you would basically perform as the text preprocessing step. The next step is tokenization. Another fundamental step where we



break down a sentence into smaller units, usually you can consider them as words.

For example, natural language processing is fun, becomes a list of individual words. This helps us analyse and count the words separately. The next is stop words, another piece of the puzzle.

You will observe that words like the, is and appear frequently in English, but they rarely add useful information for tasks like topic modelling or classification. Removing them would allow our model to focus on the words which actually carry some meaning. Another strategy which you would use here very often is stemming and lemmatisation.

We often reduce words to their root form. For example, running, ran, runs all point to the same base idea that is run. So, if I can convert these to a single base form, we can group similar words together and that reduces the vocabulary size and helps the model learn better.

Other steps might include fixing typos, removing numbers that do not carry meaning or stripping out HTML tags if our data become messy because it is extracted from the web or scraped from the web. Together, these steps do something very important, they reduce the noise in our data, and they make sure that our models are not wasting time on irrelevant or redundant information. A smaller, cleaner vocabulary would mean the model trains faster, generalises better and makes more precise predictions.

For example, in sentiment analysis, cleaning the text would mean the model focusses on words like amazing, terrible, lovely, nice that truly tells us how someone feels. In topic modelling, grouping related words makes the main topics clearer and easier to interpret. So, what we see is that text processing is not that exciting part of NLP, but it is essential.

Without it, any downstream task like embedding words, training a classifier, generating predictions, they all would become much harder and less effective. So, let us try to see these steps in action. Each of these operations brings our raw text closer to something a machine can truly learn from.

Now, we start by importing NLTK which provides us with a huge collection of tools and resources for natural language processing. NLTK, the full form is Natural Language Toolkit, one of the most popular libraries for working with human language data in Python. We can actually go to the documentation, and the documentation actually talks about the details of this library.

You would also see there is a book, but it is based on original Python 2. You can also try to explore this webpage where they kind of maintain the recent changes with the library. So, you see a good example of tokenization just we spoke about is given over here. There are several other use cases identifying named entities, displaying a parse tree etcetera.

Now, you can go to this example usage where they tend to speak about the different modules. You can explore that. For example, if I want to do just tokenization, I can say sample usage for tokenise.

It is going to show me for different sentences how the tokenization appears. So, I would want you to bookmark this for your future reference and you can explore this page in detail for you to get hold on the tools and resources for language processing. I will go back to my notebook.

Now, we are going to import stop words which is a list of common words in English that we usually remove from raw text. We are going to also import something called a porter stemmer which is an algorithm for reducing words to their root form by chopping off a part of the word. Then, we have wordnet lemmatizer which is another tool for getting the base form of a word, but it does this in an intelligent linguistically accurate way than simple stemming.



We are also going to import `re` which is a built-in python module for working with regular expressions which is useful for tasks like pattern identification, removing punctuations and special characters etcetera. Next, we have downloading some additional data that NLTK relies on. For example, the stop words resource.

It is a pre-built list of words like `the`, `is` and `which` are considered stop words in English. Then, we are also downloading `wordnet` resource which is a large lexical database which the lemmatizer would use to find the base or the dictionary form of that word. Downloading these ensures that the stop word removal and lemmatisation steps they work correctly when we run them later.

Now, we define a sample string. This text explains what text processing is and mentions some of the steps involved like removing stop words, stemming and lemmatisation. We will use this sentence to demonstrate how each preprocessing step transforms the text.

The first actual processing operation is a preprocessing operation is your `lowercase` in which converts all the letters in the text to their lowercases. Now, doing this ensures that the words like `this text` which has a capital letter, `natural language processing`, it etcetera, they all are treated as the same word by the model. So, without lowercasing, the computer would see each version with different capitalisation as a separate token which is inefficient and unnecessary.

Now, after we have done lowercasing, so this is how you can simply apply `text.lower` which converts that to lowercase, and you can print that. So, you can see this printed output which has all lowercases, simplified form. Further, that completes the first stage importing tools, downloading resources, defining your text and applying the first cleaning step.

Let us try to proceed ahead where let us see what is the next step. The next step is usually focussing on removing any punctuation marks or special characters from our text. How do we do that? We are going to use the `re` module which is python's built-in library for working with regular expressions, powerful patterns for searching and replacing text.

So, if you see this pattern here, you see this within your brackets that is a pattern which says so you see this basically is a not operator. So, I am saying not capture A to Z. So, I am saying do not capture A to Z and also this is a whitespace. I am saying apart from A to Z and whitespace, capture everything which is nothing but punctuations and special characters.

So, this pattern would match any character which is not lowercase, or a space may be periods and commas and exclamation marks, may be numbers possibly or any of the punctuations and replace that with an empty string, wherever you find that just replace that with an empty string. So, this is a step which helps us to clean the text further and most NLP models do not benefit from punctuation unless the punctuation itself carries some meaning. For example, in sentiment analysis, multiple exclamation marks might indicate some strong intense emotion, but in many tasks punctuation only adds noise.

So, once we have applied it, you can see that our sentence is free of any exclamation, commas and dots etcetera. So, it is free of that. Then the next step is to remove stop words from the text.

Stop words are very common words in a language that usually do not add meaning when we are analysing the text for tasks like classification and NLTK provides a built-in list of stop words in English. To load that you can actually load that like this. So, this is where you have the list of all stop words, and we are converting them to set for faster lookup because checking whether something is in a set is much faster than checking that in a list.

Now, we split our cleaned text into individual words using the `split` method which separates the sentence into a list of words based on spaces. Then we use a list comprehension to loop over the entire list individual words and then we do a filter if the word is not in stop



words. So, if the word that appears in the stop word list it would be filtered out.

So, for each word in the text we check whether it is not in our stop word set. If it is not a stop word we keep it, if it is a stop word we discard it. Now, this results in a new list called as filtered words that contains only the words which would carry useful meaning.

So, you can print that list and that is how you see this would be clean of any stop words. Now, as I said that the next obvious step is stemming. In this step we apply stemming to the words we kept after removing stop words.

So, what is this step? It is a process of reducing a word to its root or base form by chopping off the common endings. A very simple, but very powerful way to group related words together and that is where we start by creating an instance of this Porter stemmer. This also comes from the NLTK library, and the Porter stemmer is one of the widely used stemming algorithm in English because it uses a set of rules to systematically remove suffixes like ing, ed, ly from words.

Now, I am going to use a list comprehension to apply the stemmer to each word in our filtered word list. So, this is a list of filtered words. I am just going to run this stemmer through each word.

So, for each word the stem would return the stem the chopped version. For example, processing would become preprocess, running would become run, includes might become include etc. Removing becomes remove, stemming becomes stem, lemmatisation becomes lemma etc.

Now, what you notice is that stemming does not always produce real dictionary words, instead it produces a root form that helps group similar words together. By applying the stemming, we are simplifying the vocabulary and helping the model understand that different forms of the same word should be treated similarly which can improve the performance of your NLP models by reducing the redundancy and noise in the text data. Now, in the next and the final step of the basic text preprocessing flow, we apply lemmatisation.

Lemmatisation is similar to stemming, but instead of just chopping off endings based on simple rules, it uses a vocabulary and a grammar rule to return the true base of the dictionary form of the word, we call that as lemma. Again, NLTK provides us the algorithm, and I am going to use a WordNet lemmatizer which is which comes from WordNet which is a large lexical database of English words where the group words are together based on their meaning and grammatical relationship. So, that is how we create an instance of WordNet lemmatizer which will use this WordNet knowledge of word meanings and the part of speech to find the correct base form.

Now, I apply that in a similar manner on each word in my filtered word list. Note that we specify POS which is part of speech where I am saying that consider that verb as that word as a verb. So, I am saying treat each word as a verb because if you do so this helps the lemmatizer choose the right base form because the same word can have different lemmas depending on whether it is used as a noun or a verb or an adjective or an adverb etcetera.

So, when you do that you see that now we are producing a more linguistically accurate version of the text. See important the here important became import, which is incorrect, but important remains important. Here natural becomes n-a-t-u-r here remains natural language language.

Processing becomes process here also it becomes process includes become this here becomes this remove removing here is this here it is r-e-m-o-v-e so on and so forth. So, again this helps our NLP models understand relationships between words in a more natural



and improved quality for many tasks like classification, topic modelling, information retrieval etcetera. Now, after we clean and standardise text using preprocessing steps like lower casing etcetera the next critical step in natural language processing is tokenization.

Tokenization is nothing, but the process of breaking down a big chunk of text into smaller manageable units called as tokens. In most cases a token would be a word however, depending on the task a token could also be a sentence, a character or even a sub word unit. Now, it is essential because computers do not understand raw text the way we do right and that is where they need the text to be broken down into tokens.

Now, when we humans read a sentence our brains naturally understand where one word ends and the next begins. We do not even think about the spaces or punctuations we just read and understand. However, a computer has no such built-in intuition for a machine text is just a continuous stream of characters.

It is this tokenization which gives the machine clear boundaries allowing it to treat words or other text segments as separate units it could analyse and process. For example, if you take a sentence like natural language processing helps computers understand human language. So, a tokenizer would take it and break it into individual words like this, and this makes it possible for us to count word frequencies to build vocabularies, convert them into numbers for modelling.

So, it is actually the first step a very important critical first step which enables all other downstream task. If you want to build a word cloud you need tokens to count. If you want to build the machine learning model you first need to convert the words into numerical form.

And even modern language models like BERT and other transformers also rely on tokenization, but just in a bit more advanced way that is where they use sub word tokenization which splits the rare words into smaller manageable pieces. So, based on the task there are different levels of tokenization. Word tokenization which is most common where you split the text into words.

You also have sentence tokenization which splits a longer document into individual sentences which is useful for tasks like translation or summarization. Then you have character level tokenization which goes even granular breaking the text into individual characters and this can help handle typos, rare words, unusual spellings because the model learns patterns at a letter level. And then you have the subword tokenization which combines the strengths of words and character tokenization, and it is widely used in state-of-the-art NLP systems today.

Now, choosing how to tokenise text depends on the language you are working with, the type of data, the task you are trying to solve etcetera. So, in summary you see that tokenization is that bridge between the raw unstructured text and the structured data that a computer can work with right. Without tokenization a machine cannot learn meaningful patterns in language.

So, it is with this that we unlock the ability to analyse, classify, generate, translate, summarise etcetera text in ways that were not possible before. Now, let us try to implement that and put this idea of tokenization into action by splitting a sample text into sentences. So, we import word tokenise and sent tokenise from the LLTK library.

Word tokenise would break the text into individual words whereas, sent tokenise would break it into complete individual sentences. Interestingly we are also importing auto tokenizers from the transformers library although we are not using it just yet in the Excel it is useful later.



Video 7: Natural Language Processing- Demo Part 2

We are also going to import autotokenizer from the transformers library or although we are not going to use it just in this set, but later if you want to explore how modern transformer models handle tokenization especially at a sub word level we would be able to see that. So, we have defined a simple text natural language processing is fascinating and it has two sentences the first sentence states about the natural language processing being fascinating and the second sentence mentions that it involves tokenization and poor embeddings. Now we apply the sentence tokenization simple you can say send tokenise it is a function that takes the entire text as input and scans it for sentence boundaries usually using punctuation marks like periods or exclamation points or question marks all those valid sentence you know punctuations which indicate the end of the sentence.

Now when we run this it splits our text into a list where each element is one complete sentence. So, in our example the result is a list of two strings if you see here the first string and each string is a full sentence from the original text this is useful for tasks like summarization or translation where you may want to process or analyse each sentence individually. Now let us move ahead sometimes before we can tokenise the text using NLTK we want to make sure that certain language models or resources are available in the environment.

One of the most important resources for tokenization in NLTK is the punct tokenizer. I give you this example of right going through this page right see if you see this page would basically somewhere have you know the punct tokenizers let me see if I can search for that. So, you see here this is a punct sentence tokenizer and then you might also have at a word level, but it should be there you can search for that.

So, how do we once you have done downloading right. Now the resource is part of the underlying data that punct uses without downloading it NLTK would not be able to split text into sentences. Now we use this word tokenise on text.

Now this function again takes the entire text, but it breaks it into a list of tokens where each token is typically a word or a punctuation mark, and it does that by doing it at a more intelligent unsupervised manner. It is not like a simple split method because that only splits the text by spaces. It is a smarter it understands punctuation marks contractions and other special cases.

When we run this it involves text tokenization and word embeddings if you see here it involves text tokenization and word embeddings. It will split it into a list of word tokens and also what you notice is that the punctuation marks like the period are also treated as tokens whether you keep punctuations as tokens or remove that depends on your downstream task, but it is able to point that out or tokenise them individually. Now, so far we have seen how to split text into sentences and individual words using NLTK.

If you talk about modern natural language processing models especially transformer-based models like BERT often they use an even more advanced approach called as sub word tokenization. Sub word tokenization breaks the words down into smaller pieces called as sub words. This helps the model handle the rare or unseen words by representing them as combination of smaller known frequent occurring units.

So, we remember we imported the auto tokenizer class, and I said that we would use that for transformers. We are going to import or we are going to yeah. So, we are going to create a tokenizer by loading a pre-trained BERT tokenizer and it is called as BERT based uncased.

The term uncased means that this version of BERT converts all the text to lowercase automatically and it does not distinguish between uppercase and lowercase. Now I am



Artificial Intelligence and Machine Learning

going to again pass the entire text into your tokenizer where it is going to split a sample text into sub word units. So, now if you print that you would see that how the sub word tokens natural language processing is fascinating.

It involves text token you see here how tokenization is splitted into two and word embedding see embedding or embeddings if you see this is how it is. So, it is actually breaking it into sub word levels. That is what makes this tokenization the sub word level tokenization very effective.

That essentially completes the text preprocessing and text cleaning which is very common under natural language processing if you are building any sort of applications. Thank you.